

AUFGABE ZUR VORLESUNG

Übung 04 – Netzwerk und Datenbank

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema Docker: eigenes Netzwerk, PostgreSQL, Adminer
Dozent Dr. Ulrich Winkler
Semester 5. Semester · Informationstechnik

Vorlesung: Deck *docker-network* **Ziel:** Mehrere Container in einem **gemeinsamen Netzwerk** miteinander reden lassen. Das `log_tool` schreibt jetzt in eine echte **PostgreSQL**-Datenbank; mit **Adminer** schauen Sie in die Daten hinein.

Kontext

Bisher lief alles in einem Container. Jetzt haben wir **drei Rollen**: die Datenbank (`postgres`), ein Web-Werkzeug zum Reinschauen (`adminer`) und unser `log_tool` . Container finden sich in einem selbst angelegten Netzwerk **über ihren Namen** (Docker bringt dafür einen eingebauten DNS mit): `log_tool` verbindet sich zum Host `db` , weil der Postgres-Container so heißt.

Das `log_tool` (siehe `log_tool.py`) liest seine Verbindungsdaten aus Umgebungsvariablen: `DB_HOST` , `DB_NAME` , `DB_USER` , `DB_PASSWORD` .

Aufgabenstellung

1. Legen Sie ein Netzwerk `lognetz` an.
2. Starten Sie einen **PostgreSQL**-Container `db` in diesem Netzwerk (Datenbank `logbuch` , Benutzer `logger` , Passwort frei wählbar) mit einem Volume für die Daten.
3. Starten Sie **Adminer** im selben Netzwerk und öffnen Sie ihn auf Port 8080. Melden Sie sich an der Datenbank an (System *PostgreSQL*, Server `db`).
4. Bauen Sie das `log-tool` -Image (Dockerfile in diesem Ordner) und führen Sie es **im Netzwerk `lognetz`** aus. Fügen Sie Einträge hinzu und listen Sie sie.
5. Aktualisieren Sie Adminer – die Einträge stehen in der Tabelle `logs` . Schauen Sie sich Netzwerk und Container in Portainer an.

Tipp: Ohne gemeinsames Netzwerk findet `log_tool` den Host `db` nicht („could not translate host name“). Alle drei müssen im selben Netzwerk sein.

Musterlösung

```
cd uebungen/04_netzwerk_und_db

# 1) Netzwerk
docker network create lognetz

# 2) PostgreSQL im Netzwerk, mit Datenvolume
docker run -d --name db --network lognetz \
-e POSTGRES_DB=logbuch \
```

```
-e POSTGRES_USER=logger \  
-e POSTGRES_PASSWORD=geheim123 \  
-v pgdaten:/var/lib/postgresql/data \  
postgres:17  
  
# 3) Adminer (Web-UI) im selben Netzwerk, Port 8080  
docker run -d --name adminer --network lognetz -p 8080:8080 adminer  
# Browser -> Port 8080: System=PostgreSQL, Server=db,  
# Benutzer=logger, Passwort=geheim123, Datenbank=logbuch  
  
# 4) log_tool-Image bauen und im Netzwerk ausführen  
docker build -t log-tool:db .  
  
# Kurz warten, bis Postgres beim ERSTEN Start fertig initialisiert ist  
# (sonst: "connection refused"). Bequem mit pg_isready abwarten:  
until docker exec db pg_isready -U logger -d logbuch >/dev/null 2>&1; do sleep  
1; done  
  
docker run --rm --network lognetz \  
-e DB_HOST=db -e DB_NAME=logbuch -e DB_USER=logger -e DB_PASSWORD=geheim123 \  
log-tool:db add "Erster Eintrag in der Datenbank"  
docker run --rm --network lognetz \  
-e DB_HOST=db -e DB_NAME=logbuch -e DB_USER=logger -e DB_PASSWORD=geheim123 \  
log-tool:db list  
  
# 5) In Adminer die Tabelle "logs" öffnen -> die Einträge sind da.  
  
# Aufräumen (Volume pgdaten bleibt erhalten):  
docker rm -f db adminer
```

Was Sie gelernt haben: Container in einem gemeinsamen Netzwerk erreichen sich **über ihren Namen** (eingebauter DNS); eine Datenbank ist einfach ein weiterer Container; Adminer ist ein bequemes Fenster in die Daten. Auffällig: Es sind schon **vier** `docker run`-Aufrufe mit vielen Optionen – das vereinfachen wir in Übung 05 mit Docker Compose.